

PagUp

Autonomia e inclusione nel pagamento in contanti

Wallet educativo e accessibile per persone con disabilità cognitiva

Report di progetto

Corso LPSMT — Laboratorio di Programmazione per Sistemi Mobili e Tablet

Paolo Sarcletti · Alessandro Turri

Tecnologie: Expo (React Native) + TypeScript, Supabase

Piattaforme: Android, iOS

Anno accademico: 2025/2026

Indice

1	Introduzione	2
1.1	Contesto dell'applicazione e NEED identificato	2
1.2	Obiettivi generali del progetto	2
2	Identificazione del segmento utente	3
2.1	Utenti target	3
2.2	Perché proprio loro: la lacuna da colmare	3
3	Stato dell'arte	3
3.1	Applicazioni simili presenti negli store	4
3.2	Livello di maturazione	4
3.3	Posizionamento di PagUp	5
4	Architettura	5
4.1	Visione d'insieme	5
4.2	Diagramma dell'architettura generale	6
4.3	Modello dati e sicurezza (Row Level Security)	6
4.4	Sincronizzazione in tempo reale (broadcast)	7
4.5	Logica di dominio: l'algoritmo di pagamento	8
4.6	Flusso del wizard di pagamento	8
4.7	L'accessibilità come vincolo architetturale	8
5	Analisi critica dei possibili limiti	9
5.1	Limiti funzionali e di dominio	9
5.2	Limiti tecnici e di piattaforma	9
5.3	Limiti di sicurezza	10
5.4	Limiti di accessibilità residui	10
5.5	Sintesi	10

1. Introduzione

1.1 Contesto dell'applicazione e NEED identificato

Pagare in contanti alla cassa è un gesto che la maggior parte delle persone compie senza pensarci. In realtà è un compito cognitivo composito: bisogna riconoscere i tagli di monete e banconote, comporre un importo a partire dai pezzi disponibili nel portafoglio, valutare se la cifra posseduta è sufficiente e infine controllare il resto. Per una persona con **disabilità cognitiva** (disabilità intellettiva, sindrome di Down, disturbi dell'apprendimento) ognuno di questi passaggi può trasformarsi in un ostacolo, e l'ostacolo si presenta nel momento peggiore: in pubblico, con la fila che attende. Il risultato tipico è che l'esperienza dell'acquisto viene evitata o delegata per intero al caregiver, con un costo che non è solo pratico ma anche psicologico, perché la dipendenza per ogni acquisto erode il senso di autoefficacia della persona.

Dal lato opposto della relazione c'è il tutor — un genitore, un educatore, un operatore di centro diurno — che vorrebbe accompagnare gradualmente la persona verso l'autonomia, ma che oggi può farlo solo stando fisicamente accanto a lei. Nel momento in cui la persona esce da sola, il tutor perde ogni visibilità e ogni possibilità di intervento.

Il NEED che abbiamo identificato nasce dall'incrocio di questi due bisogni, finora mai coperti da un'unica soluzione:

1. **per lo studente:** uno strumento che guidi *passo-passo* l'atto del pagamento reale, riduca il carico cognitivo a una sola decisione per volta, dia un riscontro immediato e multisensoriale (visivo ad alto contrasto, aptico, testuale semplificato) e permetta di allenarsi senza rischi prima di affrontare la cassa vera;
2. **per il tutor:** la possibilità di configurare il portafoglio digitale, calibrare la difficoltà dell'esercizio, osservare in tempo reale come procede e ricevere subito una richiesta di aiuto (SOS) georeferenziata quando la persona è in difficoltà fuori casa.

In una frase: *trasformare un momento di stress in un'azione guidata e sicura*, rendendo la persona il più autonoma possibile senza togliere al tutor la rete di protezione.

1.2 Obiettivi generali del progetto

PagUp è un'applicazione mobile cross-platform (Android/iOS) con due ruoli distinti — studente e tutor — costruiti sopra lo stesso backend. Gli obiettivi che ci siamo posti:

- **Accessibilità come requisito di prima classe**, non come rifinitura: l'area studente rispetta il livello *AAA delle WCAG 2.1* (contrasto $\geq 7:1$ su ogni coppia testo/sfondo), con touch target da 64 dp (oltre il minimo di 48 dp), tipografia mai sotto i 18 px, etichette di accessibilità e feedback aptico su ogni azione.
- **Wizard di pagamento guidato** in quattro passi (importo → conferma → quali pezzi consegnare → esito e resto), con un algoritmo che calcola automaticamente la combinazione di monete e banconote da usare.
- **Input multimodale** dell'importo: tastierino numerico, lettura OCR del totale dallo scontrino tramite fotocamera, dettatura vocale in italiano. Chi non riesce a digitare può fotografare o parlare.
- **Modalità allenamento:** stesso gesto del pagamento, cifre generate casualmente, nessuna conseguenza sul saldo reale.
- **Supervisione remota in tempo reale:** dashboard tutor con saldo, statistiche, cronologia delle attività, editor del portafoglio e scelta della modalità di pagamento, sincronizzata via Supabase Realtime senza polling.

- **Rete di protezione:** pulsante SOS sempre visibile durante il pagamento, con invio di posizione GPS e notifica push al tutor; impostazioni dello studente protette da PIN del tutor; isolamento dei dati garantito da Row Level Security nel database.

2. Identificazione del segmento utente

2.1 Utenti target

L'applicazione è progettata attorno a una relazione uno-a-molti tra tutor e studenti. I due profili condividono il backend ma usano interfacce, temi grafici e flussi completamente separati.

	Studente (utente primario)	Tutor (utente secondario)
Chi è	Persona con disabilità cognitiva lieve o media in percorso verso l'autonomia; in prospettiva anche bambini in età scolare che imparano a gestire il denaro	Genitore, educatore, operatore di centro diurno, insegnante di sostegno
Obiettivo	Pagare e gestire il denaro in autonomia	Accompagnare, configurare, monitorare, intervenire
Interfaccia	Tema <code>studentTheme</code> : contrasto AAA, touch target 64dp, testo ≥ 18 px, una sola azione per schermata	Tema <code>tutorTheme</code> : dashboard informativa a quattro schede (home, statistiche, studenti, wallet)
Schermate	Home, pagamento guidato, allenamento	Dashboard, impostazioni (associazione via QR, gestione PIN)

Tabella 1: I due segmenti utente di PagUp e le rispettive caratteristiche.

2.2 Perché proprio loro: la lacuna da colmare

La scelta del segmento risponde a lacune precise del panorama attuale. Le app finanziarie generaliste (home banking, wallet digitali, gestori di spese) sono progettate per utenti neurotipici: presuppongono lettura fluente, astrazione numerica e tollerano interfacce dense; per il nostro profilo target risultano inaccessibili quando non ansiogene. Le app didattiche sul denaro, all'estremo opposto, trattano il denaro come concetto astratto — quiz e conteggi a schermo — ma non accompagnano la persona nel momento in cui il pagamento avviene davvero, e non prevedono alcuna figura di supervisione.

Manca cioè una soluzione che tenga insieme tre cose: un'accessibilità di livello verificabile (non “caratteri grandi” generici, ma contrasti e target misurati), un *ponte con il denaro fisico* — l'app che dice concretamente quali pezzi prendere dal portafoglio vero, mostrandone la fotografia — e una supervisione remota del caregiver. È esattamente lo spazio che PagUp occupa. Si tratta di un'utenza fragile e sottoservita, per la quale il costo dell'errore (vergogna, frustrazione, perdita di denaro) è alto e ogni dettaglio di accessibilità decide se l'app è usabile oppure no.

3. Stato dell'arte

3.1 Applicazioni simili presenti negli store

Il panorama esistente si può suddividere in quattro famiglie, nessuna delle quali copre per intero il bisogno individuato.

1. **App di riconoscimento del denaro per disabilità visiva** — ad esempio *Cash Reader* o il canale valuta di *Seeing AI* (Microsoft). Identificano una banconota inquadrata con la fotocamera e la annunciano vocalmente. Sono tecnicamente mature, ma nascono per persone cieche o ipovedenti: riconoscono *un* pezzo, non aiutano a *comporre* un importo né a decidere cosa consegnare alla cassa.
2. **App didattiche sul denaro** — esercizi di conteggio di monete ed euro pensati per la scuola primaria o per la didattica speciale. Insegnano il riconoscimento dei tagli tramite quiz, ma l'esercizio resta scollegato dall'atto reale del pagamento, l'interazione è spesso datata, la sincronizzazione cloud assente e la conformità a standard di accessibilità verificabili pressoché mai dichiarata.
3. **App di banking e wallet generaliste** — home banking, Google Wallet, Satispay e simili. Maturità tecnica molto elevata, ma fuori target: interfacce dense, gergo finanziario, denaro esclusivamente virtuale, nessuna attenzione all'accessibilità cognitiva.
4. **App di assistenza e comunicazione per la disabilità** — strumenti di CAA (comunicazione aumentativa alternativa), agende visive, app di sicurezza con pulsante di emergenza. Coprono aspetti adiacenti (comunicazione, allerta al caregiver) ma non affrontano il dominio della gestione del denaro.

3.2 Livello di maturazione

Le famiglie 1 e 3 presentano un livello di maturazione elevato, ma servono target diversi dal nostro (disabilità visiva la prima, popolazione generale la seconda). La famiglia 2, che è la più vicina per intento educativo, è anche la più debole per qualità: molte app risultano incomplete (coprono il riconoscimento ma non il pagamento), abbandonate o prive di qualunque componente di supervisione. La combinazione *didattica del denaro + accompagnamento nel pagamento reale + supervisione del caregiver in tempo reale* è, per quanto abbiamo potuto verificare negli store, semplicemente assente.

3.3 Posizionamento di PagUp

Caratteristica	Didattiche	Banking	CAA/SOS	PagUp
Accessibilità WCAG AAA	parziale	no	parziale	sì
Wizard di pagamento guidato	no	no	no	sì
Ponte col denaro fisico reale	no	no	no	sì
Modalità allenamento sicura	sì	no	no	sì
Supervisione tutor in tempo reale	no	no	no	sì
SOS con geolocalizzazione	no	no	sì	sì
Input OCR + voce	no	raro	no	sì

Tabella 2: Posizionamento di PagUp rispetto alle famiglie di applicazioni esistenti.

PagUp non compete con il banking sulla gestione finanziaria né con le app didattiche sul puro esercizio: si propone come *strumento ponte verso l'autonomia reale*, ed è l'unico prodotto della tabella a coprire l'intera filiera che va dall'allenamento protetto al pagamento vero supervisionato a distanza.

4. Architettura

4.1 Visione d'insieme

PagUp adotta un'architettura **client-serverless** di tipo BaaS (*Backend as a Service*). Il client è un'unica applicazione Expo / React Native scritta in TypeScript che, in base al ruolo dell'utente autenticato, instrada verso l'esperienza studente o quella tutor. Il backend è interamente delegato a **Supabase**, che sopra PostgreSQL fornisce autenticazione, persistenza, canali Realtime e sicurezza a livello di riga (Row Level Security). Non esiste un server applicativo intermedio: la logica di dominio vive nel client, isolata in funzioni pure e testata con Jest, mentre le regole di accesso ai dati vivono nel database sotto forma di policy dichiarative.

Livello	Tecnologia	Ruolo
Frontend	Expo SDK 54, React Native 0.81, React 19, TypeScript	UI, navigazione, logica di dominio
Stato locale	Zustand 5 + persistenza AsyncStorage	Portafoglio dello studente, sottoscrizione Realtime
Navigazione	React Navigation (Stack)	Routing per ruolo, tasto indietro su ogni schermata non-home
Backend	Supabase (PostgreSQL)	Auth, dati, Realtime, RLS
Notifiche	Expo Notifications + Expo Push API	Avvisi push al tutor (pagamenti, SOS)
Servizi nativi	ML Kit (OCR), Speech Recognition, Location, Camera, Haptics	Input multimodale, SOS, feedback tattile

Tabella 3: Stack tecnologico di PagUp.

4.2 Diagramma dell'architettura generale

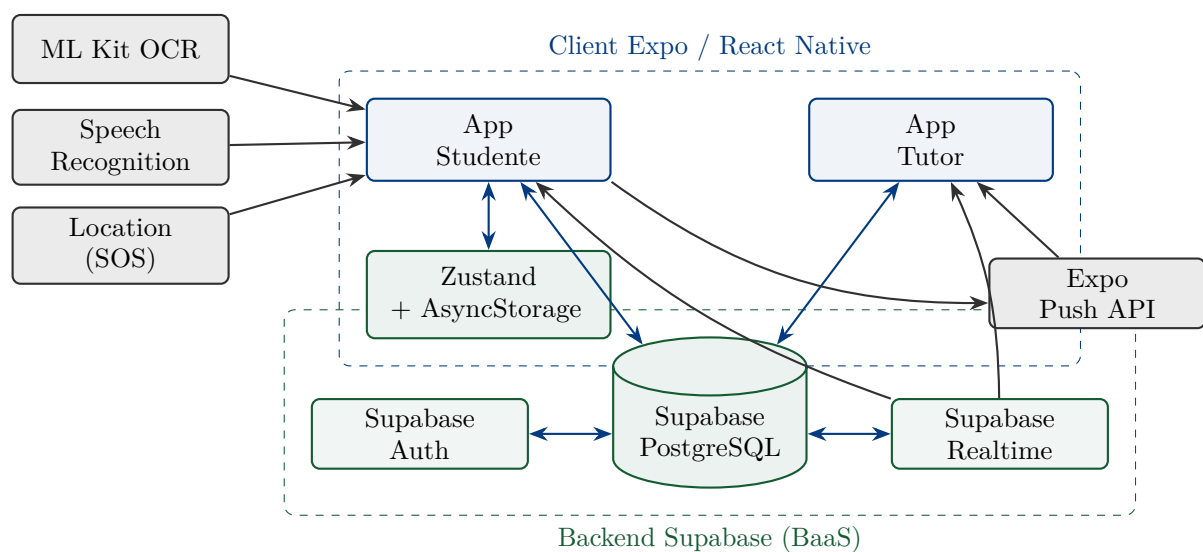


Figura 1: Architettura generale: un unico client a doppio ruolo, servizi nativi del dispositivo, backend Supabase e canale di notifiche push.

L'associazione tra un tutor e uno studente avviene di persona e senza scambio di credenziali: lo studente mostra un QR code generato dall'app (contenente il proprio identificativo), il tutor lo scansiona dalla schermata impostazioni e la riga corrispondente viene creata nella tabella ponte. Da quel momento le policy RLS aprono al tutor la visibilità sui dati di quello studente, e soltanto su quelli.

4.3 Modello dati e sicurezza (Row Level Security)

Il database è composto da cinque tabelle. La sicurezza non è affidata al client ma *dichiarata nel database*: ogni utente autenticato può leggere e scrivere solo le righe che le policy gli concedono, in funzione di `auth.uid()` e della tabella ponte `tutor_students`.

Tabella	Contenuto e regole principali
<code>profiles</code>	Estende <code>auth.users</code> : ruolo, username, nome completo, <code>tutor_pin</code> . Creata da trigger al signup. Lo studente legge i profili dei propri tutor; il tutor quelli dei propri studenti.
<code>wallets</code>	Un portafoglio per studente; campo <code>items</code> JSONB con l'array di <code>MoneyItem</code> (<code>id</code> , <code>value</code> , <code>type</code> , <code>imageUri</code>). Lo studente aggiorna il proprio; il tutor legge e scrive quelli dei propri studenti.
<code>tutor_students</code>	Tabella ponte della relazione tutor-studente; contiene la modalità di pagamento assegnata (<code>payment_mode</code> : <code>exact/fast</code>). Popolata tramite la scansione del QR.
<code>activity_logs</code>	Cronologia eventi: <code>payment</code> , <code>sos</code> , <code>wallet_adjustment</code> . Per i pagamenti registra importo richiesto, importo consegnato e modalità usata; per gli SOS la schermata di provenienza, l'importo eventualmente inserito e la posizione GPS (in <code>metadata</code> JSONB).
<code>push_tokens</code>	Token Expo Push per utente e piattaforma, usati per recapitare gli avvisi al tutor.

Tabella 4: Schema dati e regole di accesso (RLS) di PagUp.

4.4 Sincronizzazione in tempo reale (broadcast)

Il punto qualificante dell'architettura è la sincronizzazione bidirezionale *senza polling* tramite Supabase Realtime, che ritrasmette ai client sottoscritti gli eventi `postgres_changes` del database. Quando il tutor ricarica il portafoglio di uno studente, la riga `wallets` cambia, l'evento `UPDATE` raggiunge il canale a cui il dispositivo dello studente è sottoscritto, l'inventario locale si aggiorna e parte un feedback aptico che avvisa la persona senza bisogno di leggere nulla. Simmetricamente la dashboard del tutor è sottoscritta agli eventi su `wallets` e `activity_logs`: un pagamento o un SOS compaiono sullo schermo del tutor nel momento in cui avvengono. Lo stesso meccanismo sblocca in automatico la home dello studente nell'istante in cui il tutor ne scansiona il QR.

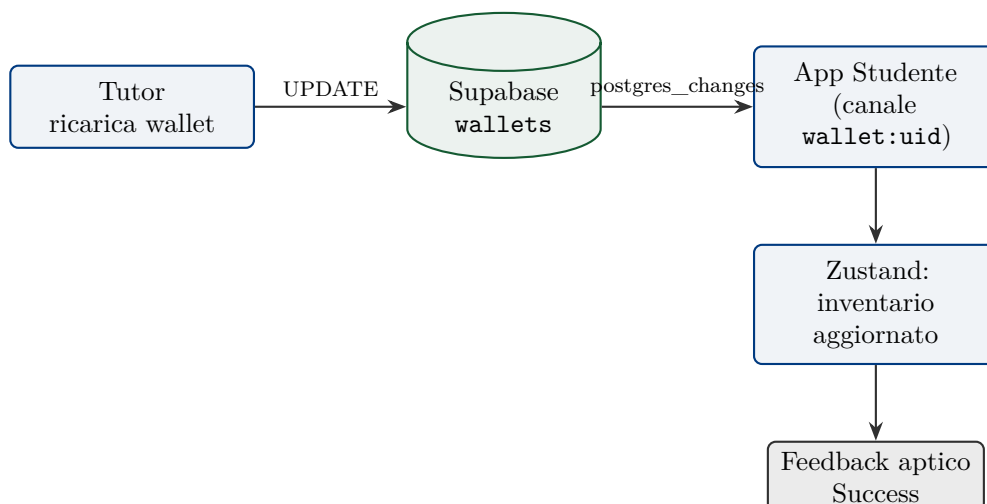


Figura 2: Flusso di broadcast Realtime: la modifica del tutor si propaga al dispositivo dello studente senza polling.

4.5 Logica di dominio: l'algoritmo di pagamento

Il cuore funzionale è il calcolo di *quali pezzi* consegnare per coprire un importo. Tutti i conti avvengono in **centesimi interi**, per evitare gli errori di arrotondamento del floating point, e l'importo obiettivo viene arrotondato per eccesso al multiplo di 10 centesimi: i tagli da 1, 2 e 5 centesimi sono esclusi sia dall'algoritmo sia dall'editor del tutor, perché maneggiarli aggiunge complessità senza valore didattico. Le modalità sono due, e le sceglie il tutor per ciascuno studente:

- **Precisa (exact):** cerca prima una combinazione *esatta* dell'importo (variante di subset-sum con memoizzazione delle somme raggiungibili in centesimi); se non esiste, ripiega sulla più piccola combinazione che copre l'importo, minimizzando il resto.
- **Veloce (fast):** privilegia la semplicità cognitiva scegliendo il *singolo* taglio più piccolo che da solo copre l'importo (per pagare 3,67 euro l'app propone la banconota da 5 euro). È il gradino iniziale del percorso: prima si impara “dai una banconota abbastanza grande e aspetta il resto”, poi si passa alla composizione precisa.

La logica di dominio è isolata in funzioni pure (`src/Utils/paymentLogic.ts`, `smartExtract.ts`) e coperta da test unitari Jest (combinazione esatta, fallback, modalità veloce, esclusione delle monete piccole, casi di insufficienza), secondo la regola di progetto “hooks e utility (logica) separati dai componenti (UI)”.

4.6 Flusso del wizard di pagamento

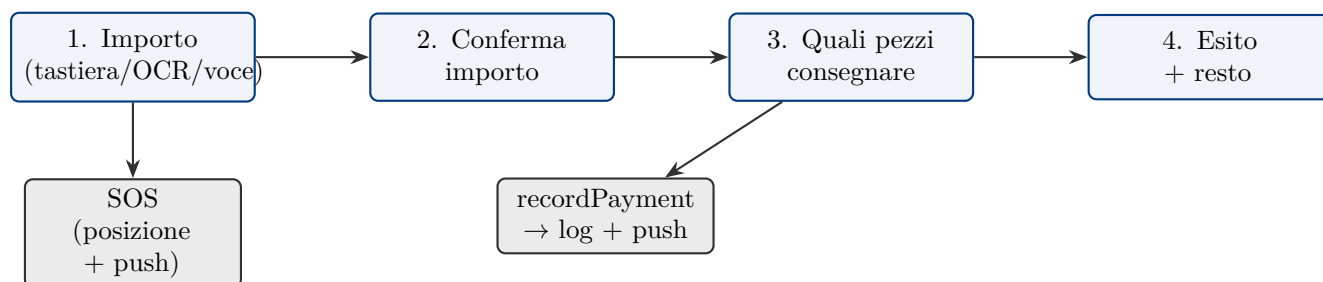


Figura 3: Macchina a stati del PaymentWizard: ogni passo è una schermata a sé, con pulsante SOS sempre disponibile e registrazione dell'evento alla conferma del pagamento.

Ogni passo occupa l'intero schermo e chiede una sola decisione; si avanza e si torna indietro con animazioni direzionali coerenti (avanti = da destra, indietro = da sinistra) che aiutano a mantenere l'orientamento nel flusso. Nel terzo passo l'app mostra le *fotografie* dei pezzi da prendere dal portafoglio reale. Alla conferma, `processRealPayment` sottrae i pezzi dall'inventario, persiste il wallet su Supabase, registra un `activity_log` per ciascun tutor associato — con importo richiesto, importo consegnato e modalità usata — e invia la notifica push. La schermata finale, a sfondo verde pieno, comunica l'esito e il resto atteso con un linguaggio ridotto all'essenziale.

4.7 L'accessibilità come vincolo architetturale

L'accessibilità non è uno strato cosmetico ma una scelta d'architettura: l'app definisce due *design system* distinti come moduli TypeScript tipizzati (`src/theme/student.ts` e `tutor.ts`), e nessuna schermata usa colori o dimensioni fuori dai token del proprio tema. Il tema studente dichiara nel codice, accanto a ogni colore, il rapporto di contrasto verificato con la formula di luminanza relativa delle WCAG (dal 7,0:1 minimo del testo disabilitato al 21:1 del nero su bianco); fissa il touch target a 64 dp con distanza minima di 16 dp tra bersagli adiacenti, e il corpo

del testo a 18px con interlinea 1,6. Ogni elemento interattivo espone `accessibilityLabel` e `accessibilityHint` in italiano semplice, e ogni azione produce un feedback aptico differenziato (selezione, conferma, successo, avviso, errore), così che l'esito di un gesto sia percepibile anche senza leggere. Il tema tutor, rivolto a un utente neurotipico, adotta il livello AA e una densità informativa maggiore: la separazione dei due temi permette di ottimizzare ciascuna esperienza senza compromessi reciproci.

5. Analisi critica dei possibili limiti

Pur coprendo il bisogno individuato, l'applicazione ha limiti noti. Li analizziamo nel dettaglio insieme alle direzioni di miglioramento, distinguendo ciò che è una scelta di scope da ciò che è un debito tecnico.

5.1 Limiti funzionali e di dominio

- **Il resto è solo indicativo.** L'app calcola quanto resto attendersi ma non può verificare che il cassiere lo restituisca correttamente: l'anello finale dell'interazione reale resta fuori dal controllo del software. *Estensione:* un passo opzionale di verifica del resto, in cui la persona compone i pezzi ricevuti e l'app conferma che il conto torna.
- **Allenamento non adattivo.** Le cifre della modalità allenamento sono generate casualmente da un insieme di tagli semplici, senza memoria dei successi e degli errori della persona. *Estensione:* difficoltà progressiva guidata dallo storico, con obiettivi assegnabili dal tutor.
- **Solo valuta Euro e lingua italiana.** Tagli, fotografie e parser vocale sono cablati sull'Euro e sull'italiano (`EURO_DENOMINATIONS`, `moneyImages.ts`, numerali in `useVoiceInput`). *Estensione:* astrazione della valuta e internazionalizzazione dei testi.

5.2 Limiti tecnici e di piattaforma

- **Notifiche push limitate ad Android.** `registerForPushNotifications` restituisce `null` su iOS: un tutor con iPhone non riceve gli avvisi, incluso l'SOS (l'evento resta comunque visibile in dashboard via Realtime). *Estensione:* configurazione APNs e build di distribuzione con EAS.
- **OCR e voce richiedono la build nativa.** ML Kit e Speech Recognition non esistono in Expo Go: i moduli sono caricati con `require` difensivo e, in loro assenza, l'app ripiega sul solo tastierino. *Estensione:* distribuzione esclusiva via dev-client o store.
- **Il pagamento reale richiede connettività.** Il portafoglio è persistito localmente (`AsyncStorage`) e consultabile offline, ma la conferma di un pagamento scrive su Supabase: senza rete l'operazione fallisce con un avviso. *Estensione:* coda locale di eventi con riconciliazione al ritorno della connessione.
- **Costo computazionale dell'algoritmo.** La ricerca delle somme raggiungibili è pseudo-polinomiale nel numero di pezzi e nell'importo in centesimi; con inventari molto grandi potrebbe degradare. Nella pratica l'inventario di un esercizio è piccolo; un limite esplicito di cardinalità metterebbe il vincolo nero su bianco.
- **Scalabilità del Realtime.** La dashboard del tutor si sottoscrive agli eventi sulle tabelle `wallets` e `activity_logs` filtrando lato client, e i log letti sono limitati alle ultime 300 righe. Con molti utenti conviene spostare il filtro lato server e paginare.

5.3 Limiti di sicurezza

- **PIN del tutor in chiaro.** Il campo `tutor_pin` è memorizzato come testo nel profilo: è una barriera contro l'apertura accidentale delle impostazioni da parte dello studente, non un meccanismo di sicurezza forte. *Estensione:* hashing del PIN o biometria con `expo-local-authentication`, già tra le dipendenze del progetto.
- **Invio push dal client.** Lo studente legge i token dei tutor associati e chiama direttamente l'Expo Push API: funzionale ma esposto ad abusi. *Estensione:* spostare l'invio in una Edge Function Supabase autorizzata.
- **QR di associazione senza scadenza.** Il codice mostrato dallo studente è statico e l'associazione non richiede conferma reciproca. *Estensione:* codici a tempo e doppio consenso.

5.4 Limiti di accessibilità residui

- L'app è costruita per la disabilità cognitiva (e, di riflesso, serve bene anche l'ipovisione grazie a contrasti e dimensioni), ma non affronta esplicitamente le **disabilità motorie gravi** (switch access, scansione) né è stata validata in modo sistematico con gli screen reader di sistema. *Estensione:* campagna di test con TalkBack e VoiceOver e supporto a input alternativi.
- Manca una **lettura vocale in uscita** (text-to-speech) delle istruzioni: oggi l'output è visivo e aptico ma non parlato, mentre per utenti con difficoltà di lettura la voce sarebbe il canale più diretto.
- Soprattutto, manca una **validazione sul campo con utenti reali**: le scelte di accessibilità rispettano gli standard, ma solo una sperimentazione con persone del target e i loro educatori può confermare che il carico cognitivo percepito sia davvero quello progettato.

5.5 Sintesi

La maggior parte dei limiti elencati discende da scelte di scope coerenti con un progetto didattico — valuta unica, lingua unica, push solo Android, sicurezza del PIN proporzionata alla minaccia reale — più che da difetti d'impianto. L'architettura scelta (logica in funzioni pure testate, autorizzazioni dichiarate nel database con RLS, sincronizzazione a eventi) fa sì che le estensioni descritte — verifica del resto, allenamento adattivo, push iOS, coda offline, Edge Function, internazionalizzazione — si possano innestare in modo incrementale, senza rimettere mano alle fondamenta.